Beyond the HBM Wall:

Coherent-Memory Oversubscription for AI Workloads on NVIDIA Grace Hopper

Mohamed Abdelnaby and **Wu Feng**
{mohamedabdelnaby, wfeng}@vt.edu

Department of Computer Science, Virginia Tech, USA

55th International Conference on Parallel Processing (ICPP'26)
State of Practice in Deploying Supercomputers with NVIDIA Superchips Workshop

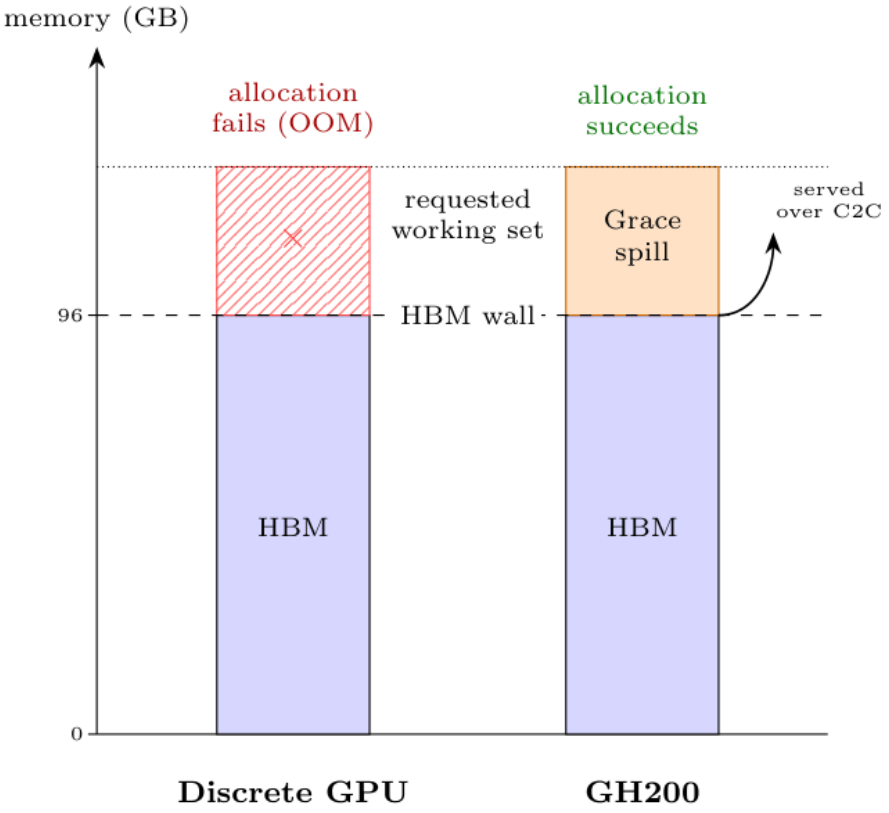
VIRGINIA TECH

SyNeRG
synergy.cs.vt.edu

Motivation: The HBM Wall in NVIDIA GH200 Superchip

- HBM capacity of GH200? 96 GB
- Working set > 96 GB?
  - ***Discrete GPU:*** No overflow path, so allocation results in an *out-of-memory (OOM)* error
  - ***GH200 Superchip:*** The coherent CPU+GPU memory holds the overflow in Grace and *streams it over NVLink-C2C*
    - Issue: The *completed run* is ***SLOWER***, sometimes significantly slower, than an in-HBM run

The HBM wall becomes a performance limiter. Is the slower run worth the extra node-hours?



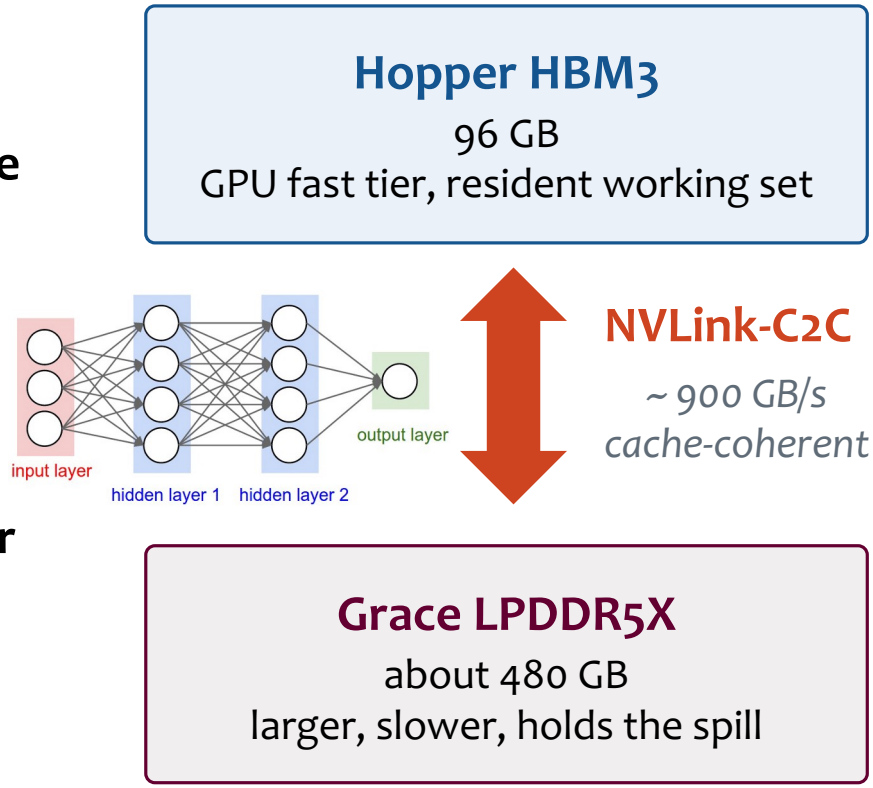
Contributions

1. **First characterization of AI training under coherent-memory oversubscription on the NVIDIA GH200 Superchip**

   Throughput characterization of AI training *beyond the HBM wall*

2. **A framework-agnostic memory interposer**

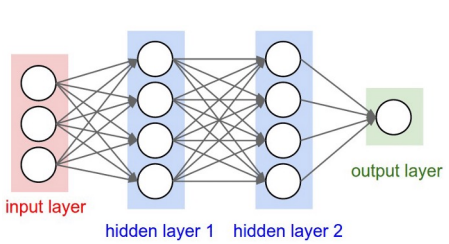
   An evaluation of the "ChAI: Chapel+AI" framework on the NVIDIA Superchip



Background: Grace Hopper Memory Hierarchy

- Coherent address space spans two physically separate memories
  - Hopper HBM3: 96 GB at ~ 4 TB/s
  - Grace LPDDR5x: 480 GB via NVLink-C2C at 900 GB/s (450 GB/s per direction)
- NVLink-C2C link
  - Cache-coherent, so a `cudaMallocManaged` allocation can live in either memory and migrate pages on access
- Benefit: HBM oversubscription
  - Resident HBM stays ~ 96 GB *while* the logical memory allocation grows beyond HBM with overflow held in Grace LPDDR5X

Hopper HBM3
96 GB
GPU fast tier, resident working set



NVLink-C2C
~ 900 GB/s
cache-coherent

Grace LPDDR5X
about 480 GB
larger, slower, holds the spill

Related Work

Work	Overflow	Link	Workloads
DRAGON	**transparent**	PCIe, NVMe (non-coherent)	HPC, **AI**
Ganguly et al.	**transparent**	PCIe, simulated (non-coherent)	HPC
Allen & Ge	**transparent**	PCIe (non-coherent)	HPC
Schieffer et al.	**transparent**	**C2C (coherent)**	HPC
Lin & Jeon	**transparent**	PCIe (non-coherent)	**AI training**
SuperOffload	explicit	**C2C (coherent)**	**AI training**
This Work	**transparent**	**C2C (coherent)**	**AI training**

Our AI training workload? **ChAI: Chapel+AI**

Workload: ChAI: Chapel+AI*



ChAI 1.0: Bind to LibTorch C++

- Leverage the same AI kernels that PyTorch does
- Result
  - Chapel front-end (for ChAI 1.0) outperforms Python front-end (for PyTorch)

Capability	ChAI 1.0	PyTorch
Inference	✓	✓
Training (autograd, optimizers)	✓	✓
Persistent tensor handles	✓	✓
Optimizers (Adam, SGD, fused)	✓	✓
Loss functions	✓	✓
CNN and vision layers	✓	✓
Transformer and attention	✓	✓
Recurrent (LSTM, GRU)	✓	✓
CPU and GPU device API	✓	✓
CUDA Graphs	✓	✓
Operator fusion	✗	✓
Mixed precision (BF16, FP8)	◐	✓
Load PyTorch checkpoints	✓	✓
Tensor backend	LibTorch	LibTorch

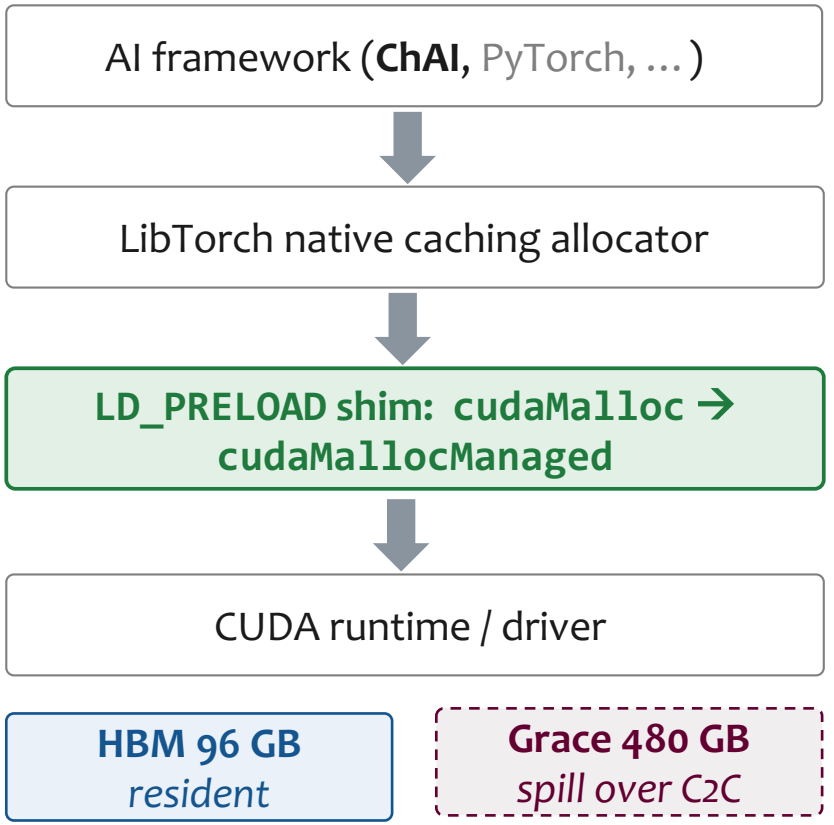
✓ supported ◐ partial or experimental ✗ absent

* Abdelnaby, Sathre, Feng, "To ChAI or Not to ChAI: Chapel+AI for Training and Inference," *30th IEEE High-Performance Extreme Computing*, Sept. 2026

Approach: Framework-Agnostic Memory Interposition

- Interposition at the CUDA-runtime ABI, *not* at a framework's API (*a la* PyTorch)
  - A `LD_PRELOAD` shim redirects every `cudaMalloc` to `cudaMallocManaged` (with `cudaMemAttachGlobal`)
  - The framework reaches device memory via LibTorch's caching allocator, which draws all backing store from `cudaMalloc`

```
cudaMalloc(ptr, size) becomes
cudaMallocManaged(ptr, size, cudaMemAttachGlobal)
```



Workload Suite

Configuration	Varied Axis	Dominant Footprint	Design Role
GPT (irregular)	vocabulary	embedding table	few rows read per step
GPT (regular)	batch size	feed-forward activations	whole tensor re-read, same model
ResNet-152	image resolution	2D activation maps, dense	dense 2D convolution
3D denoiser	batch size	3D feature maps, dense	higher compute per byte than ResNet-152

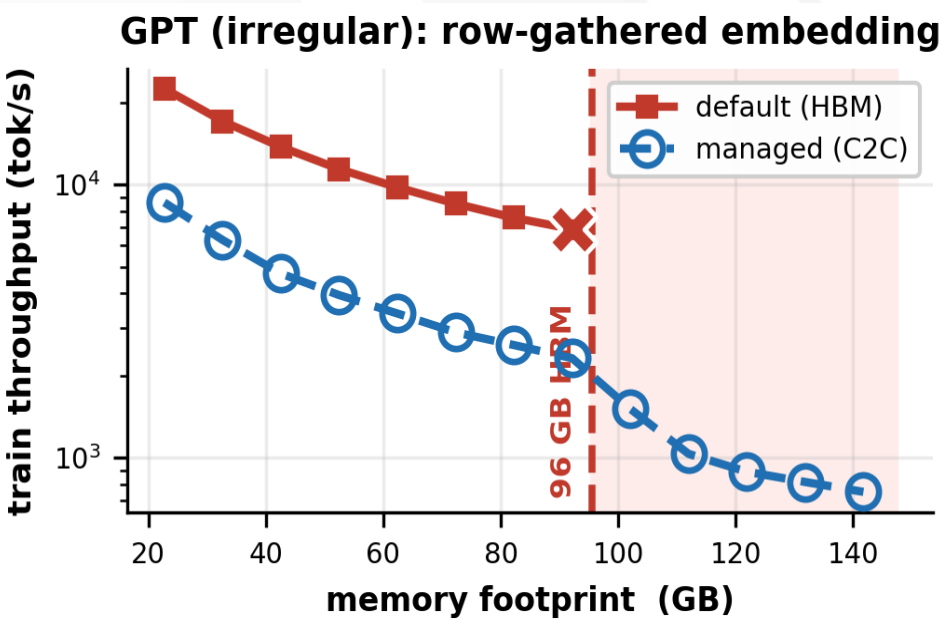
- The two GPT rows hold the model fixed & *vary* how a step reads the spilled tensor.
- The two convolution rows, ResNet-152 and 3D denoiser, hold the read pattern regular & *vary* arithmetic intensity.

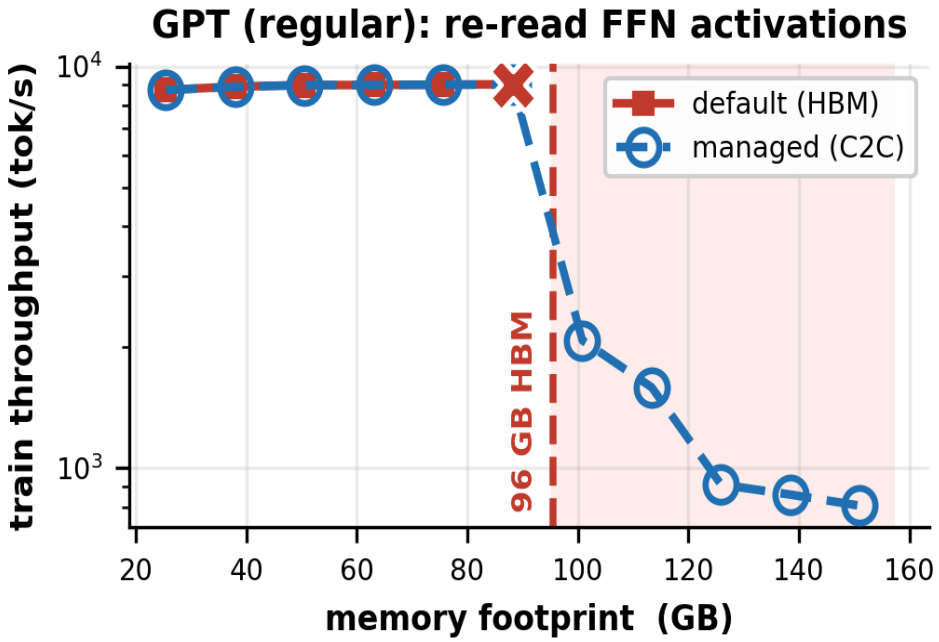
"irregular" = a step accesses a small, data-dependent subset of the tensor

GPT (irregular) reads a few thousand embedding rows out of 1.4 million per step

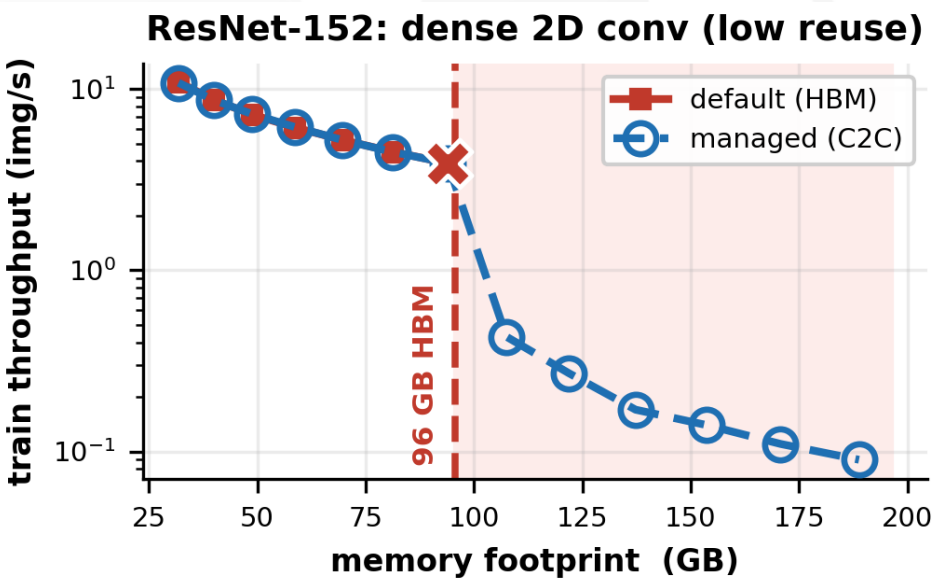
GPT (regular) re-reads its ***entire*** FFN activation tensor every backward pass.

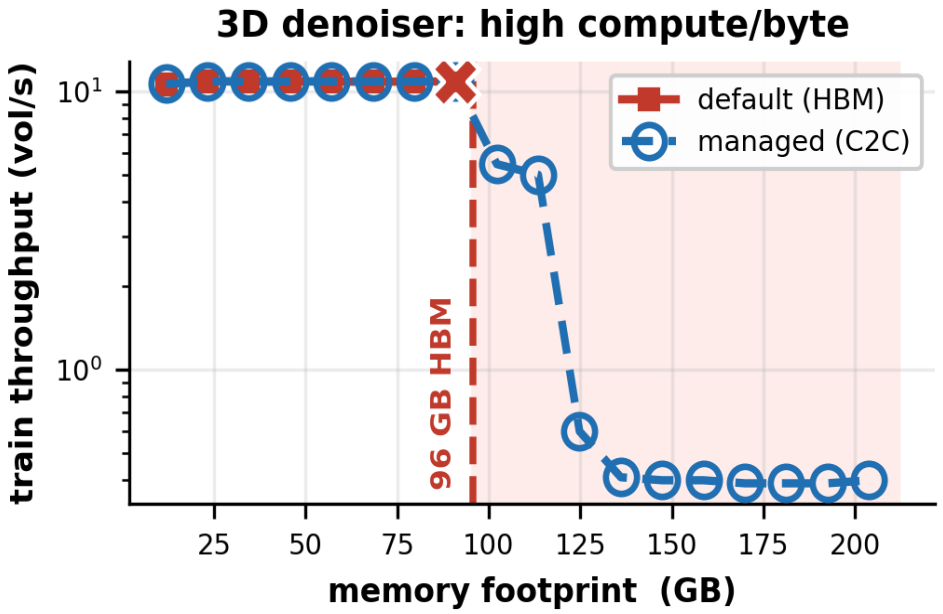
Results: Training Throughput for GPT (tok/s)



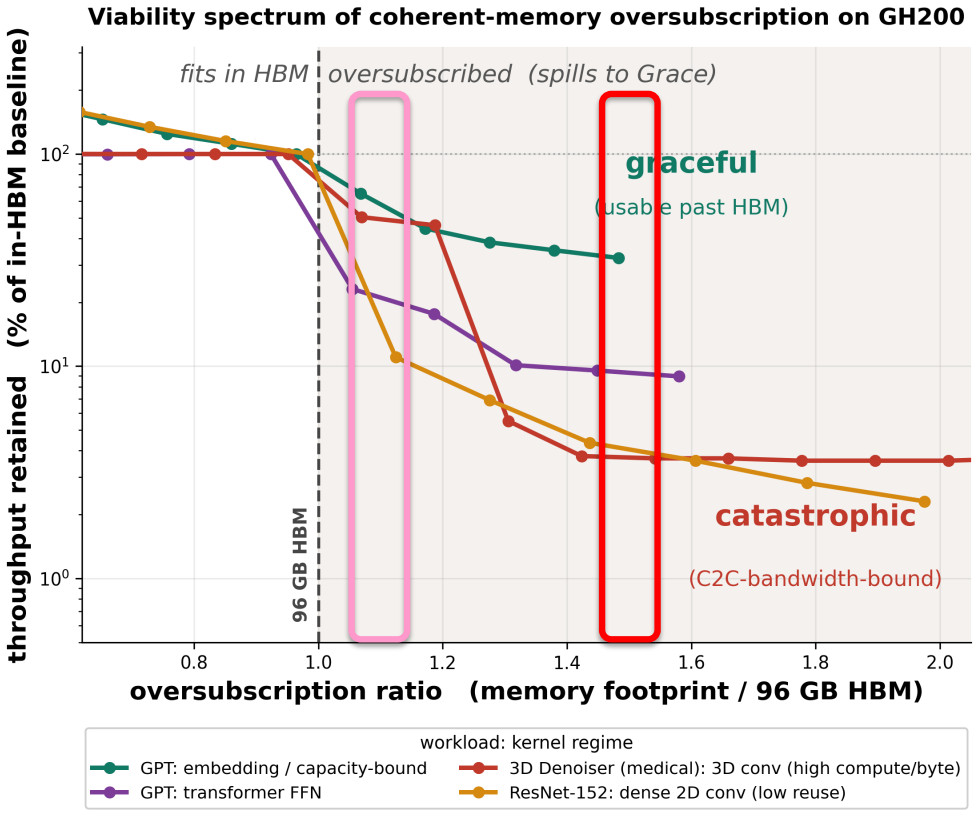


Results: Training Throughput for Imaging





Results: Viability Spectrum for Memory Oversubscription



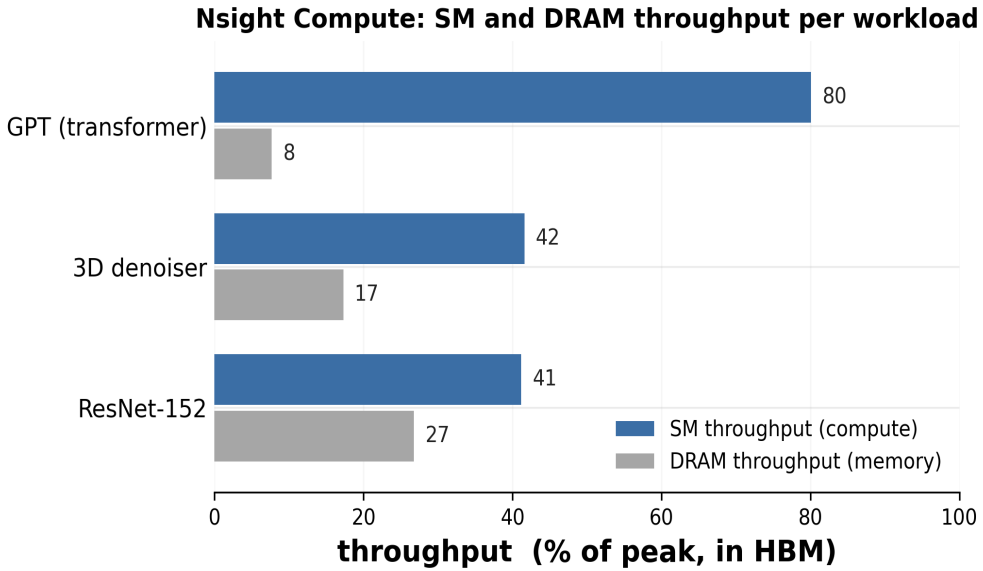
Workload	HBM Cost	Thru-put @ ~1.1x	Thru-put @ ~1.5x	Performance Degradation
GPT (irregular)	2.9x	65%	32%	graceful
GPT (regular)	1.0x	23%	10%	moderate
3D denoiser	1.0x	50%	4%	moderate to catastrophic
ResNet-152	1.0x	11%	4%	catastrophic

HBM Cost is the in-HBM *managed* tax.

Throughput @ ~1.1x and @ ~1.5x

- Percentage (%) of throughput retained at 1.1x and 1.5x oversubscription ratios, respectively.
- Caveat: Since the AI workloads are different, each step adds a different amount of GB, so the ratios will *not* be exactly aligned @ 1.1x and 1.5x.

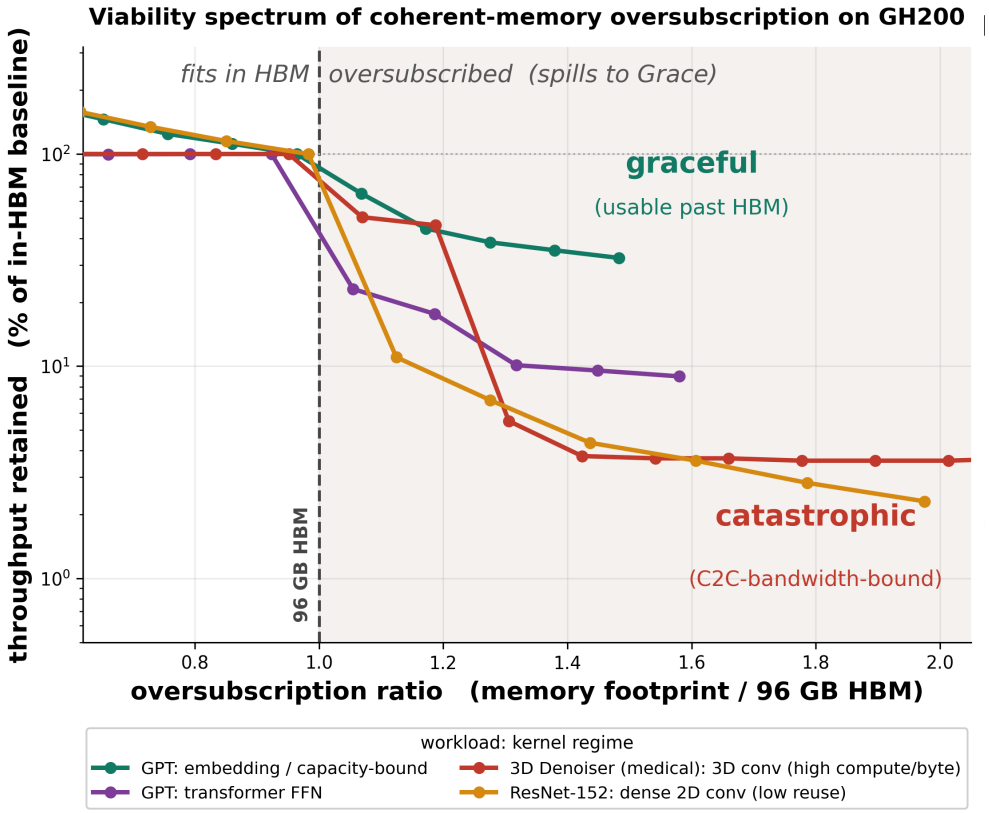
Analysis: Performance Degradation (Graceful vs. Moderate vs. Catastrophic)



In-HBM Profile
- GPT compute-bound (SM 80%, DRAM 8%)
- ResNet-152 more balanced (SM 41%, DRAM 27%)

Performance degradation from memory oversubscription *influenced by*
- Boundedness of AI training
  - Compute-bound: Graceful-to-moderate degradation
  - Memory-bound: Moderate-to-catastrophic
- "Bytes spilled per step"
  [How much data each step pulls over C2C]
  - Fewer bytes (e.g., sparse lookup for GPT irregular)
  - Whole tensor re-load (e.g., GPT regular)

Analysis: Performance Degradation (Graceful vs. Moderate vs. Catastrophic)



Performance degradation from memory oversubscription *influenced by*

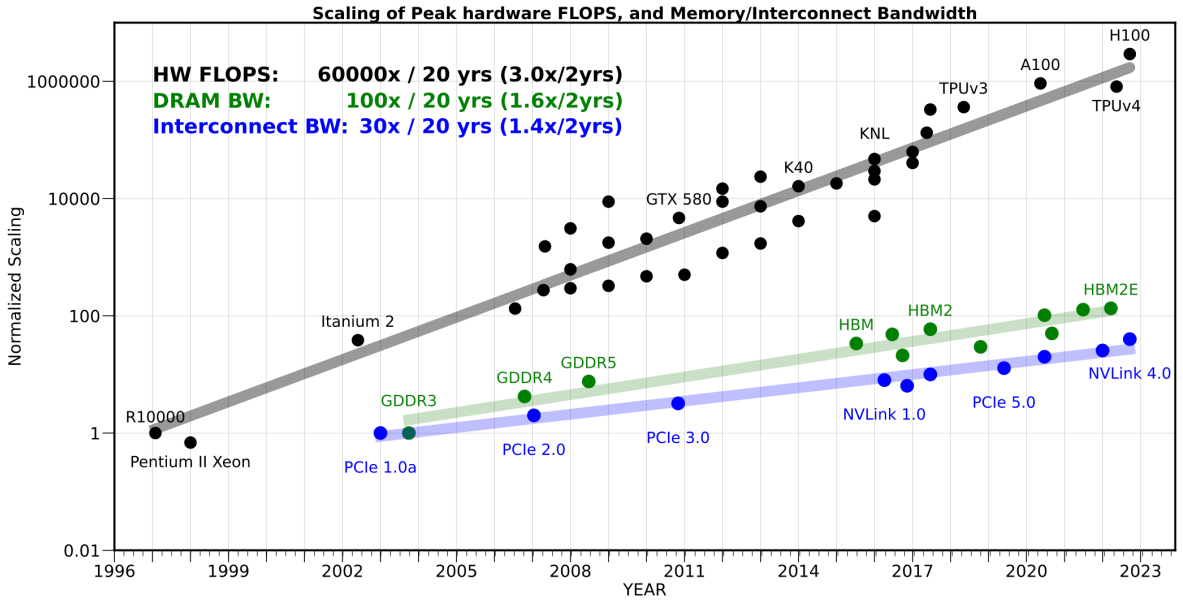
- Boundedness of AI training
  - Compute-bound: Graceful-to-moderate degradation
  - Memory-bound: Moderate-to-catastrophic
- "Bytes spilled per step"
  [How much data each step pulls over C2C]
  - Fewer bytes (e.g., sparse lookup for GPT irregular)
  - Whole tensor re-load (e.g., GPT regular)

Looking Forward

Trends

- Scaling of FLOPS and Memory/Interconnect Bandwidth
  "AI and Memory Wall" by Gholami et al., UC Berkeley, March 2024.

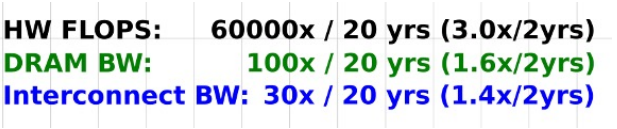
HW FLOPS: 60000x / 20 yrs (3.0x/2yrs)
DRAM BW: 100x / 20 yrs (1.6x/2yrs)
Interconnect BW: 30x / 20 yrs (1.4x/2yrs)



Looking Forward

Trends

- Scaling of FLOPS and Memory/Interconnect Bandwidth
  "AI and Memory Wall" by Gholami et al., UC Berkeley, March 2024.



- Scaling of Superchip Architecture and Memory/Interconnect Bandwidth

Architecture	CPU Pairing	Memory Type	Memory Capacity (per GPU)	Memory Bandwidth	CPU-to-GPU Coherent Bandwidth (NVLink-C2C)
Grace Hopper	Grace CPU	HBM3 / HBM3e	141 GB / 282 GB	4 TB/s	900 GB/s bidirectional
Grace Blackwell	Grace CPU	HBM3e	192 GB / 288 GB	8 TB/s	900 GB/s bidirectional
Vera Rubin	Vera CPU	HBM4	288 GB	13 – 22 TB/s	1800 GB/s bidirectional

- Scaling of AI Models
  - Memory Footprint? 4x – 5x annual growth
    → 16x – 25x growth every 2 years
  - Memory Bandwidth? 4x annual growth
    → 16x growth every 2 years

Implication: Scaling "beyond the HBM wall" will become *increasingly important*!

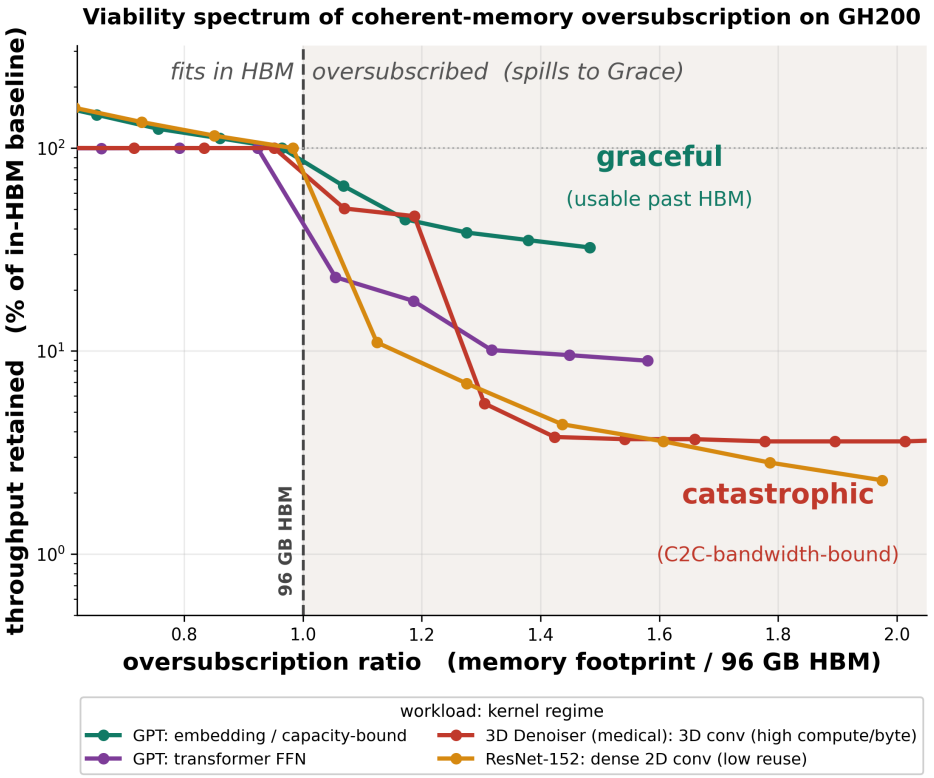
Conclusion: Beyond the HBM Wall on NVIDIA Superchip

Coherent chip-to-chip (C2C) memory enables AI training of models to run *beyond the HBM wall*, albeit with varying levels of performance degradation

Scaling *beyond the HBM wall* is become increasingly important based on the trajectories of architectural and AI trends

memory/interconnect BW
~ 1.5x / 2 years

memory/interconnect BW
> 16.0x / 2 years



Thank you. Questions?